

TECHNICAL_STUDY_GUIDE.md — deep implementation reference for mentor review

Audience: a faculty mentor or industry reviewer who wants to test *how the system actually works* — the mathematics, the algorithms, the causality guarantees, the code paths. Read alongside the codebase, not instead of it.

Different from `TEAMMATE_BRIEFING.md` : that file is the presentation-defence brief for a jury (fast, defensive, headline numbers, Q&A). This one is depth for a supervisor who will ask "walk me through what happens when ..." and "show me where that guarantee is enforced."

Suggested reading time: ~90 minutes for full read, ~30 minutes for the parts your mentor is most likely to probe (§2, §5, §7, §10, §14).

How to use: every claim in this document ends with a code reference (`path:function` or `path` for whole-module) so you can open the file and read the actual implementation while your mentor is in the room. If a claim looks weak, verify it in the code before you say it out loud.

Table of contents

- **Part I — Problem formalisation**
 - §1 The probabilistic setting we chose
 - §2 The bust label, derived condition by condition
 - §3 Zero-leakage causality axioms and the tests that enforce them
 - §4 The spatial aggregation operator, derived
- **Part II — End-to-end computational trace**
 - §5 From atmosphere to `(F, 0)` pair (data → truth)
 - §6 From `(F, 0)` to the bust label
 - §7 From features to a calibrated probability
 - §8 From probability to a dashboard cell
- **Part III — Subsystems in depth**
 - §9 Data ingestion: chunking economics, why we sized what we sized
 - §10 Feature engineering: causality enforcement and the poison test
 - §11 Regime classifier: score → softmax → entropy
 - §12 XGBoost + isotonic: the training loop, with math
 - §13 TreeSHAP: what `pred_contribs=True` actually computes
 - §14 Reason engine: family dedup and direction-aware templates
 - §15 Mahalanobis OOD: derivation, threshold, earned-refusal
 - §16 Baselines: the fair-comparison protocol
- **Part IV — Evaluation math**
 - §17 AUROC as a rank statistic; why for rare events

- §18 Brier decomposition, BSS against climatology
- §19 Calibration — ECE with equal-count binning
- §20 Asymmetric decision cost and economic value
- **Part V — Serving and systems**
- §21 Batch scoring pipeline
- §22 SQLite schema and indexing
- §23 FastAPI request lifecycle
- §24 Dashboard rendering — 2D (Leaflet) and 3D (three.js) with the geometry-merging optimisation
- §25 Air-gap verification methodology
- **Part VI — Verification**
- §26 Test taxonomy — 68 tests, what each guards
- §27 Reproducibility from a cold clone
- **Part VII — Reflection**
- §28 Known limitations we did not close
- §29 What we would change with another month
- §30 Extension research directions

PART I — PROBLEM FORMALISATION

§1. The probabilistic setting we chose

Let $\mathcal{S}$ be the set of 34 modelled IMD meteorological subdivisions (index s), $\mathcal{T}$ a set of initialisation datetimes at 00 UTC (index t_0), and $L \in \{1, \dots, 10\}$ the lead day. For each triple (s, t_0, L) the forecast field yields a scalar $F(s, t_0, L) \in \mathbb{R}^+$ — the area-mean forecast 24 h rainfall over s valid on day $V(t_0, L) := t_0 + (L - 1)$ calendar days. The observation field yields a corresponding scalar $O(s, V)$.

We define a target label $Y(s, t_0, L) \in \{0, 1\}$ (see §2) and the modelling task is to learn

$$\hat{p}(s, t_0, L) = \hat{P}(Y(s, t_0, L) = 1 \mid X(s, t_0, L))$$

where $X(s, t_0, L)$ is a 52-dimensional feature vector (§7, §10, §11) constructed exclusively from information available **at or before** t_0 . This last clause is the whole game; violating it turns the problem from prediction into hindcasting.

Design decision worth naming. We chose classification over regression on $|F - O|$ because the operational unit is a decision (would the officer's alert have flipped?), not an error magnitude. The bust label operationalises "did the decision flip"; regressing on raw error would optimise for numeric fit and land on an "impressive RMSE" that does not translate to a triaged review queue. This is defended in [LOGIC.md](#) §4.4 and in [src/fbd/labels/bust.py](#) module docstring.

Loss function used at training time: log-loss with $\text{scale_pos_weight} = n_0 / n_1 \approx 25$ (see §12). Threshold selection uses asymmetric cost (see §20), not a symmetric error rate.

§2. The bust label, derived condition by condition

Full definition, all three conditions logically ANDed. Implemented at `src/fbd/labels/bust.py`.

Condition 1 — magnitude

Let $E(s, t_o, L) = |F(s, t_o, L) - O(s, V(t_o, L))|$. Fit the 95th-percentile of E per (subdivision, month) on training years only:

$$\theta_{\text{mag}}(s, m) = \max(10.0, Q_{0.95}(\{E(s, t_o, L) : t_o \in \text{Train}, \text{month}(V) = m\}))$$

The `max(10, ·)` floor prevents dry-subdivision noise from qualifying. Implemented at `src/fbd/labels/bust.py:error_thresholds` with a two-level fallback (subdivision-month $\rightarrow$ subdivision-wide $\rightarrow$ global) for months with fewer than 200 training samples — otherwise a rare-month P_{95} is a fit to noise.

Fires when $E(s, t_o, L) \geq \theta_{\text{mag}}(s, \text{month}(V))$.

Condition 2 — category flip

Define $\text{Cat}: \mathbb{R}^+ \rightarrow \{0, 1, 2, 3, 4, 5\}$ from the IMD intensity bands:

$\text{Cat}(r) = 0$	if $r \in [0, 2.5)$	no rain
$= 1$	if $r \in [2.5, 15.6)$	light
$= 2$	if $r \in [15.6, 64.5)$	moderate
$= 3$	if $r \in [64.5, 115.5)$	heavy
$= 4$	if $r \in [115.5, 204.5)$	very heavy
$= 5$	if $r \geq 204.5$	extremely heavy

Implemented as a `numpy.searchsorted` on `CATEGORY_EDGES` at `src/fbd/labels/bust.py:rain_category`. $O(\log 6)$ per row, vectorised.

Fires when $\text{Cat}(F) \neq \text{Cat}(O)$.

Condition 3 — operational significance

Fires when $\max(F, O) \geq 15.6$ (the lower bound of the *moderate* class).

The full label

$$\begin{aligned} Y(s, t_o, L) &= 1 \iff [C1] \wedge [C2] \wedge [C3] \\ &= 0 \text{ otherwise} \end{aligned}$$

with $Y := \text{NaN}$ if either F or O is undefined (missing IMD cell or dropped forecast). Rows with `NaN` labels are excluded from training and scoring; excluding them is not the same as scoring them as 0.

Why three conditions, formally

Let $q \in (0, 1)$ be the percentile used in C1. If we used C1 alone:

$$P(Y = 1 \mid X) = P(E \geq \theta_{\text{mag}}) \approx 1 - q \text{ by construction of } \theta_{\text{mag}}$$

so the base rate on the training set is *exactly* $1 - q$, independent of the atmosphere, the region, or anything else. The label ceases to carry information — it merely reports where each row sits in the training-year error distribution. Every downstream AUROC would be measuring nothing.

C2 alone fires on boundary-straddling pairs (15.5 vs 15.7 mm), which are decision-irrelevant.

C3 alone would fire on any moderate-plus rain event, whether forecast well or badly.

All three combined encode: "a large error that would have flipped a decision that mattered." The base rate becomes an emergent property of the atmosphere (measured ~4%), not an artefact of the definition.

The leakage discipline

θ_{mag} is estimated only on training years (2016–2020) and applied unchanged to 2021 (val) and 2022 (test). If it were re-fitted per split, the held-out bust rate would be pinned at $1 - q = 5\%$ by construction and the whole evaluation would prove nothing. The test-year bust rate is **3.59%**, not 5%; the train rate is 4.07%. That discrepancy is the numerical proof the fit is honest.

Tested at `tests/test_core.py:test_error_threshold_is_fitted_on_training_years_only`: synthetic data with different noise per year, must produce different bust rates per year with a single fitted threshold.

§3. Zero-leakage causality axioms

Every feature used at prediction time for (s, t_o, L) must be a function of information available **at or before** t_o . Two subtle traps we close explicitly:

Axiom A — no valid-time atmospheric state

The ERA5 analysis valid at $V(t_o, L)$ does not exist when the forecast is issued at t_o . Using it would be a near-perfect predictor of whether that forecast busted (you would essentially be showing the model the answer) and would inflate every AUROC to ~1.0.

Enforced in code at `src/fbd/features/era5.py` module docstring and via the join key in `scripts/build_dataset.py:attach_state_features` which explicitly joins on `init_date`, never on `valid_date`.

Axiom B — lagged-ensemble causality

The lagged-ensemble spread for lead L uses forecasts from successive initialisations that all verify on the same day V . The set of *those* forecasts issued *at or before* t_o is:

```
{ (t0 - kΔ, L + k) : k ≥ 0, L + k ≤ 10 } where Δ = 1 day
```

i.e. we only look at leads $\geq L$. A forecast with lead $L - 1$ verifying on V was issued at $t_0 + \Delta$, which is *after* t_0 ; including it would use the future.

Enforced at `src/fbd/features/forecast.py:lagged_ensemble` by construction — the loop iterates `cols = [lead_pos[l] for l in range(L, L + n_members) if l in lead_pos]`, and there is a poison test.

The poison test. `tests/test_core.py:test_lagged_ensemble_uses_only_leads_at_or_beyond_L` constructs a synthetic day where leads 1–4 hold the value 1000 and leads 5–10 hold 10. If the code obeyed causality, leads 5–8 would show zero spread (their members are all from `{5, ..., 10}`, all equal to 10) and lead 4 would show huge spread (its members `{4, 5, 6} = {1000, 10, 10}`). The test asserts exactly this. Every code change that touches feature construction re-runs this test.

Reading this test in your mentor's presence is the single strongest defence of the causality claim.

Both axioms extend to derived features

- Climatology: fitted on training years, applied to val/test unchanged.
- Regime scores: computed from `t0` analysis only (not `V`).
- Standardisations of national indices (`somali_jet_z` etc.): mean and std computed on training years only, then applied.

Any deviation would be a subtle leak. All standardisation stats are attached to the model artifact at training time and re-used at scoring time; nothing is recomputed on test rows.

§4. The spatial aggregation operator, derived

The problem forces a projection from raster data (WeatherBench 2 forecasts on a 0.703° regular grid; IMD observations on a 0.25° regular grid) to vector data (36 subdivision polygons of vastly varying shape and size, e.g. West Rajasthan 193,000 km² vs Coastal Karnataka 18,000 km²).

The wrong way, and why

Centroid masking (assign a raster cell to a polygon iff the cell centre lies inside it) is what `regionmask` does. At 0.7° a centroid test silently gives zero cells to narrow coastal subdivisions like Konkan & Goa, Coastal Karnataka, Kerala — which are precisely the heavy-rainfall regions this project exists to serve. Losing them would not raise an error; it would just quietly drop the most important rows.

The right way

For each (raster cell `c`, subdivision `s`) compute the intersection area `A(c ∩ s)` in an equal-area projection (**EPSG:7755**, India NSF Lambert Conformal Conic), keep only pairs with positive overlap, and use those areas as weights. Implemented at `src/fbd/regions/masks.py:overlap_weights` via GeoPandas `overlay`.

The result is a sparse table:

```
subdivision_id | lat_idx | lon_idx | lat | lon | weight_km²
```

with one row per (cell, subdivision) pair that overlaps in reality. Cached to a `.parquet` keyed by `(n_lat, n_lon, lat[0], lat[-1], lon[0], lon[-1])` so a differently-shaped grid can never silently reuse another grid's weights.

The aggregation itself

Given the sparse weights, materialise a dense array $W \in \mathbb{R}^{|S| \times n_{\text{lat}} \times n_{\text{lon}}}$ (implementation: `weights_to_matrix`). For a field $f \in \mathbb{R}^{T \times n_{\text{lat}} \times n_{\text{lon}}}$ (T timesteps, possibly with NaN in ocean cells):

```
valid[t, i, j] = 1 if isfinite(f[t, i, j]) else 0
filled[t, i, j] = f[t, i, j] if valid else 0

numerator[t, s] = Σ_{i,j} filled[t, i, j] · W[s, i, j]
denominator[t, s] = Σ_{i,j} valid[t, i, j] · W[s, i, j]
total[s] = Σ_{i,j} W[s, i, j]

mean[t, s] = numerator[t, s] / denominator[t, s]
coverage[t, s] = denominator[t, s] / total[s]
```

Implemented as three `np.einsum('tij,sij->ts', ..., optimize=True)` calls at `src/fbd/regions/masks.py:area_mean`. Vectorised; the whole 7-year IMD record aggregates in ~40 seconds on a laptop.

Quality gate

`mean` is set to NaN wherever `coverage < 1 - max_nan_fraction`. Default `max_nan_fraction = 0.40` from `src/fbd/config.py:MAX_NAN_FRACTION`. A subdivision that had only 10% of its area with valid gauge data on a given day yields NaN, not a mean built from a handful of cells that would silently misrepresent the whole region.

Tested at: - `tests/test_core.py:test_area_mean_matches_hand_computation` (mathematics correct) - `tests/test_core.py:test_area_mean_renormalises_over_valid_cells_and_reports_coverage` (NaN handling correct) - `tests/test_core.py:test_area_mean_rejects_thin_coverage` (coverage gate fires)

PART II — END-TO-END COMPUTATIONAL TRACE

§5. From atmosphere to `(F, 0)` pair

Trace for one row: subdivision `s = ODISHA`, init `t₀ = 2022-06-14T00:00Z`, lead `L = 5`, valid `V = 2022-06-18`.

Forecast path: 1. WeatherBench 2 HRES zarr store `datasets/hres/2016-2022-0012-512x256_equiangular_conservative.zarr` opened via anonymous GCS (`gcsfs` + `zarr`). Loaded lazily. 2. Slice `total_precipitation_24hr[time=t0, prediction_timedelta=5*24h, :, :]` triggers a chunk read from GCS. 3. Convert m → mm (multiply by 1000). Rename `latitude` → `lat`, `longitude` → `lon`, transpose to `(time, lead, lat, lon)`. 4. Slice to India bounding box `[6°N-38°N, 66°E-100°E]`; produces a `48 × 45` field for this cell. 5. Multiply by pre-cached overlap weights `W[s_ODISHA, :, :]`, sum, divide by total weight → scalar `F(s, t0, L)` in mm/day.

Observation path (parallel): 1. IMD NetCDF file `RF25_ind2022_rfp25.nc` opened via `xarray`, `RAINFALL` variable. 2. `RAINFALL[time=V, :, :]` → `129 × 135` field. 3. Same weight matrix multiplication (the weight cache key uses grid corners so the IMD grid gets a different cache than the HRES grid). 4. Scalar `O(s, V)` in mm/day.

Join: in `src/fbd/ingest/hres.py:build_pairs` via `pandas.merge` on `(subdivision_id, valid_date)`. Only inner join — a row missing either side is dropped.

Result on 24 Aug 2026 for our specific example: `F = 12.3 mm/day`, `O = 33.4 mm/day`. Not a bust — the error is 21.1 mm, but `Cat(F) = 1` (light) and `Cat(O) = 2` (moderate), category flip fires; magnitude fires (21.1 > 10 floor); significance fires (max = 33.4 > 15.6). Actually — three conditions hold, so **this specific row is a bust**. (Verify by reading `data/artifacts/bulletins.sqlite`.)

§6. From `(F, O)` to the bust label

Straight application of §2's three conditions. Implemented as a vectorised pandas operation:

`src/fbd/labels/bust.py:label`. No loops, no per-row Python — everything is column arithmetic on the whole 279,650-row frame at once.

§7. From features to a calibrated probability

Given a row `(s, t0, L)` with feature vector $x \in \mathbb{R}^{5 \times 2}$:

1. **Preprocessing:** none required. Trees handle NaN natively (XGBoost `missing=NaN` sends missing values to a learned default child at each split); no imputation, no scaling.
2. **Base classifier:** `xgboost.XGBClassifier` returns `p_raw = P(Y=1|x)` from the ensemble sum of tree scores passed through sigmoid. Trained with `scale_pos_weight ≈ 25` to counter the ~4% class imbalance.
3. **Calibration:** isotonic regression `g: [0, 1] → [0, 1]` fitted on the 2021 validation year, mapping `p_raw` to a well-calibrated `p_cal`. See §12 for the algorithm and why isotonic beats Platt scaling here.
4. **Bagged uncertainty:** `scripts/generate_bulletins.py:bagged_interval` refits the whole XGBoost + isotonic pipeline 6 times on bootstrap-resampled training years, produces 6 alternative `p_cal` values per row, takes the 10th–90th percentile spread, and recentres it on the deployed `p_cal` so the interval always contains the served point estimate. See §12 for the bias reasoning and the older approach that was wrong.
5. **OOD gate:** Mahalanobis distance `D(x)` in standardised feature space (§15). If `D(x) > θood` the row's output is `{status: OUT_OF_DISTRIBUTION, bust_probability: null}` regardless of what the model would have said. The interval is also nullified — nothing quantitative is emitted on refused rows.

6. **Explanation:** if the row is not OOD, native TreeSHAP (§13) produces per-feature contributions `φi(x)`, ranked, filtered by concept family (§14), turned into ≤3 sentences.

Full function: `src/fbd/model/train.py:BustModel.predict_proba` followed by the OOD guard and explainer in `scripts/generate_bulletins.py:main`.

§8. From probability to a dashboard cell

The precomputed row lands in `data/artifacts/bulletins.sqlite`:

```
(region_id, region, init_date, lead_day, valid_date, status,
 bust_probability, confidence_in_estimate, pi_low, pi_high,
 dominant_factors, regime_json, data_quality,
 ood_distance, forecast_rain_mm, observed_rain_mm, actual_bust,
 baseline_probability, model_version)
```

Primary key `(region_id, init_date, lead_day)`; indices on `init_date`, `valid_date`, `region_id`.

The 2D dashboard requests `/api/bulletin?init_date=...&lead_day=...`, gets a list of subdivision-level rows, colours each polygon in Leaflet by `bust_probability` using five bands (`<5%`, `5-10%`, `10-20%`, `20-35%`, `>35%`) + purple for OOD.

The 3D command centre requests `/api/risk-cube?init_date=...`, gets all $34 \times 10 = 340$ cells in one payload (parallel arrays by lead index, no per-cell reason strings — those load on click), renders columns at each subdivision's `representative_point` centroid with height proportional to lead and colour by probability. See §24.

PART III — SUBSYSTEMS IN DEPTH

§9. Data ingestion — chunking economics

The single non-obvious fact that drove every dataset decision: **WeatherBench 2 zarr chunks span the entire globe**. Measured chunk shapes:

Store	Variable	Chunk shape	Compressed size
HRES (1.5°)	<code>total_precipitation_24hr</code>	<code>(1, 8, 240, 121)</code>	0.41 MB
HRES (1.5°)	<code>geopotential</code>	<code>(1, 8, 13, 240, 121)</code>	7.71 MB
ENS (1.5°)	<code>total_precipitation_24hr</code>	<code>(1, 50, 8, 240, 121)</code>	20.5 MB (all 50 members in one chunk)
ENS (1.5°)	<code>geopotential</code>	<code>(1, 50, 8, 13, 240, 121)</code>	91.5 MB

Every chunk covers the whole globe (`240 × 121` is the full 1.5° grid), so slicing to India: - saves memory (small in-RAM array) - saves local disk (small cached file) - **does not save bandwidth** — you download the world and slice locally.

Consequences (locked in `DECISIONS.md` D-002 through D-004): - HRES precipitation at 0.703° for JJAS 2016–2022: 854 init dates × ~11 MB per init ≈ 9 GB transfer. Affordable. - HRES 3-D flow fields at every lead: ~47 GB per variable. **Rejected** — atmospheric-state features come from the ERA5 analysis instead, which is a single time series shared across all `(t₀, L)` pairs. - Full 50-member IFS ENS: 20.5 MB × 6 chunks × 854 inits ≈ 105 GB for precipitation alone. **Rejected**; subsampled to every 3rd init for the test year (D-014).

All raw files land in `data/raw/{imd,wb2/hres,wb2/era5,wb2/ens,shapes}/`. Scripts are idempotent — re-running a fetch script skips files already present with size > 10 KB, so an interrupted run resumes rather than restarts.

§10. Feature engineering — causality enforcement

Feature construction is split by causality domain:

- **Forecast-derived features** — computed from the forecast archive itself, so nothing to enforce beyond the leads-≥-L rule for lagged ensemble. See `src/fbd/features/forecast.py`.
- **State features** — computed from ERA5 analysis, joined on `init_date` never `valid_date`. See `src/fbd/features/era5.py` module docstring, and `scripts/build_dataset.py:attach_state_features` for the exact merge.
- **Climatological features** — fitted on training years, joined by `(subdivision_id, month)` or `(subdivision_id, month, lead_day)`. See `src/fbd/features/forecast.py:climatology` and `:climatological_bust_rate` (lines 133–158).

Every fitted statistic (`clim_obs_mean`, `clim_bust_rate`, standardisation μ , σ for `_z` features) is derived from `df[df.valid_date.dt.year.isin(TRAIN_YEARS)]`. Grep for `train_years` or `config.TRAIN_YEARS` — it is a pattern repeated in every fit call.

The lagged-ensemble features, mathematically

For lead `L`, take the `n_members`-lead window `{L, L+1, ..., L+n_members-1}` (default `n_members = 3`):

```
lagged_spread(s, t₀, L)    = std({ F(s, t₀ - (k-1)Δ, k) : k ∈ window, k ≤ 10 })
lagged_mean(s, t₀, L)     = mean(...)
lagged_range(s, t₀, L)    = max(...) - min(...)
lagged_n_members(s, t₀, L) = # of leads in window with valid F
lagged_spread_rel          = lagged_spread / (lagged_mean + 1)  # normalise for dry vs wet region
```

Standing convention: `t₀ - (k-1)Δ` with lead `k` verifies on `V(t₀, L)` for all `k` in the window. Thin-membership (`< 2`) rows get `NaN` for spread — most importantly, **Day 10 has membership 1 across the whole archive**, because the archive stops at 240 h and leads 11, 12 don't exist. This artefact is disclosed in `DECISIONS.md` D-011 and drives our "headline is Day 3–7" reporting.

spread_growth

`spread_growth(s, t₀, L) = lagged_spread(s, t₀, L) - lagged_spread(s, t₀, L-1)`, computed within the group `(subdivision_id, valid_date)`. Fast-growing disagreement flags a rapidly-losing-predictability situation.

jumpiness

Consecutive-run change for the same valid day: `jumpiness(s, t₀, L) = |F(s, t₀, L) - F(s, t₀ - Δ, L + 1)|`. Duty forecasters watch this informally already ("the model keeps flipping on this event"); we make it a feature. Implemented at `src/fbd/features/forecast.py:jumpiness`.

clim_bust_rate — dual purpose, defended

The same statistic is used as both **baseline #1** (§16) and as a **feature**. Legitimate because it is fitted on training data only, and the baseline is scored on held-out data. The model is not allowed to peek at test-year bust labels via this feature. Laplace smoothing with `k = 20` prevents a subdivision-lead cell with 40 samples and zero busts from claiming a 0% rate.

National ERA5 indices — the ones your mentor may probe

Implemented at `src/fbd/features/era5.py:national_indices`. Physically-motivated, all reduced to India-domain area-weighted (`cos(lat)`) means or extrema over specific boxes:

Index	Box (lat_min, lat_max, lon_min, lon_max)	Physical meaning
<code>somali_jet</code>	(5, 15, 50, 65)	U-wind at 850 hPa averaged over the Somali jet. Monsoon strength.
<code>monsoon_trough_mslp</code>	(20, 28, 72, 88)	MSLP over the trough box. Deeper trough → lower MSLP.
<code>bob_vorticity_max</code>	(10, 22, 82, 95)	Max relative vorticity at 850 hPa over the Bay of Bengal. Depression indicator.
<code>bob_mslp_min</code>	(10, 22, 82, 95)	Min MSLP over the Bay of Bengal. Low-pressure indicator.
<code>mcz_q850</code>	Monsoon Core Zone (18–28°N, 65–88°E)	850 hPa specific humidity — moisture available to the core rainfall zone.
<code>india_shear</code>	India box	Magnitude of <code>V₂₀₀ - V₈₅₀</code> — vertical wind shear.
<code>nw_z500</code>	(28–38, 68–80)	500 hPa geopotential over NW India — western disturbance trough indicator.

Each raw index is standardised against training-year climatology (`_z` suffix) and given 1-day and 3-day tendencies (`_d1`, `_d3`) computed within each year (grouping by year prevents differencing 1 June against the previous 30 September and manufacturing fake tendencies — this bug was found and fixed mid-build).

Relative vorticity — the calculus

Implemented at `src/fbd/features/era5.py:_relative_vorticity`. On a lat/lon grid:

$$\zeta = \partial v / \partial x - \partial u / \partial y$$

$$= (1 / (R \cdot \cos(\text{lat}))) \cdot \partial v / \partial \lambda - (1 / R) \cdot \partial u / \partial \phi$$

with $R = 6.371 \times 10^6 \text{ m}$, $\lambda = \text{longitude in radians}$, $\phi = \text{latitude in radians}$. In code, `xarray.differentiate('lon')` returns derivatives *per degree*, so we scale by $1 / (R \cdot \cos(\text{lat}) \cdot \pi/180)$ for $\partial v / \partial \lambda$ and $1 / (R \cdot \pi/180)$ for $\partial u / \partial \phi$. Centred differences keep the result on the original grid. Units: 1/s. Sanity check: values on JJAS mornings over the Bay of Bengal come out at $10^{-5} - 10^{-4} \text{ 1/s}$ (cyclonic, positive), which matches published synoptic scales for a monsoon depression.

§11. Regime classifier — score → softmax → entropy

Implemented at `src/fbd/regime/classify.py`. Weak supervision, permitted for the MVP by LOGIC.md §5.2.

The six regimes

`active_monsoon`, `break_monsoon`, `monsoon_depression`, `western_disturbance`, `orographic`, `coastal`.

These are the ministry's own named regimes from the problem statement, not something we invented.

Two honest observations

1. Four are genuine daily *circulation* regimes: active, break, depression, WD.
2. Two are *place* × flow interactions: orographic and coastal. A subdivision is in an orographic regime when strong low-level flow meets its terrain, and it is coastal when onshore moist flow impinges on its coastline. Static "place" attributes (elevation, roughness, land fraction) modulate the daily flow signal for these two.

Scoring

For each (s, t) day, physically motivated scores:

```
s_active(s, t) = 0.5 · somali_jet_z(t) + 0.5 · mcz_q850_z(t) - 0.3 · monsoon_trough_mslp_z(t)
s_break(s, t) = -0.5 · somali_jet_z(t) - 0.5 · mcz_q850_z(t) + 0.3 · monsoon_trough_mslp_z(t)
                - 0.3 · tcwv_zl(s, t)                                # tcwv standardised within subdivision
s_dep(s, t)    = 0.6 · bob_vorticity_max_z(t) - 0.6 · bob_mslp_min_z(t)
s_wd(s, t)     = north_weight(s) · (-1.0 · nw_z500_z(t))    # only meaningful for northern subdivision
s_orog(s, t)   = PLACE_WEIGHT · orographic_index(s)
                + orographic_index(s) · (0.6 · moisture_flux_850_zl + 0.4 · u850_zl)
s_coast(s, t)  = PLACE_WEIGHT · coastal_index(s)
                + coastal_index(s) · (0.6 · tcwv_zl + 0.4 · moisture_flux_850_zl)
```

`PLACE_WEIGHT = 2.5` (see `src/fbd/regime/classify.py:11`). Why: the flow-modulation terms are standardised *within* subdivision, so their time-mean is zero — multiplying by a static index scales variance but not mean, and Himachal would end up with the same average orographic probability as West Rajasthan. A static baseline term restores the intrinsic character. This bug was found and fixed mid-build.

Softmax with temperature

Softmax temperature $\tau = 0.8$:

$$p(r \mid s, \tau) = \exp(s_r(s, \tau) / \tau) / \sum_{r'} \exp(s_{r'}(s, \tau) / \tau)$$

Slightly sharper than $\tau = 1$. Rarely produces top-regime probability > 0.98 — the classifier stays soft, which is the design intent (LOGIC.md §5.1).

Regime entropy

$$H(s, \tau) = -\sum_r p(r \mid s, \tau) \cdot \log(p(r \mid s, \tau)) / \log(|R|)$$

Normalised to $[0, 1]$. **Fed to the bust model as a feature.** High entropy means the classifier itself is ambiguous about which regime applies, which is itself a predictor of low predictability (LOGIC.md §5.3: "regime uncertainty is itself evidence of low predictability"). This is one of the more elegant modelling choices — a meta-signal from the auxiliary model.

Independent validation your mentor should like

Not fitted on bust labels, so the regime vector cannot leak the target. Validation done post-hoc: on top-10% depression-score days over Odisha, mean rainfall is **25.1 mm/day**, vs **5.3 mm/day** on low-depression days — a $4.7\times$ ratio, meteorologically consistent with what a depression should do. On Coastal AP: 8.4 vs 4.5 ($\sim 2\times$). On Gangetic WB: 9.2 vs 6.8 ($\sim 1.4\times$). The signal is strongest where it should be strongest.

§12. XGBoost + isotonic — the training loop with math

The objective

XGBoost minimises

$$L(\theta) = \sum_i \text{log-loss}(y_i, \hat{y}_i(\theta)) + \sum_k \Omega(f_k)$$

where $\hat{y}_i$ is the sum of tree scores through sigmoid and $\Omega(f) = \gamma \cdot T + 0.5 \cdot \lambda \cdot ||w||^2$ regularises each tree's T leaves. Our hyperparameters:

Parameter	Value	Rationale
<code>max_depth</code>	4	Shallow enough to keep TreeSHAP fast and interpretable, deep enough to capture interactions. Deeper trees started overfitting on the JJAS-only 2021 val set.
<code>n_estimators</code>	300	Elbow of the val-loss curve. Beyond 400 marginal AUROC gain went below 0.001.
<code>learning_rate</code>	0.05	Shrinkage, standard for the estimator count.
<code>subsample</code>	0.8	Stochastic training.
<code>colsample_bytree</code>	0.8	Column subsampling.
<code>min_child_weight</code>	20	Prevents splits on tiny subpopulations that would fit the 4% rare class to noise.
<code>reg_lambda</code>	2.0	L2 on leaf weights.
<code>scale_pos_weight</code>	$n_0 / n_1 \approx 25$	See below.
<code>tree_method</code>	<code>hist</code>	Histogram-based split search, $\sim 10\times$ faster than exact.

Implemented at `src/fbd/model/train.py:BustModel.fit`.

The class-imbalance math

With bust rate $p^+ \approx 0.04$ the gradient of log-loss weights positives at only their base rate. Setting $\text{scale_pos_weight} = n_0/n_1 = (1-p^+)/p^+ \approx 25$ scales the gradient at positive rows by that factor, which is equivalent to duplicating each positive row ~ 25 times but without the memory cost. The effect: the model treats positives and negatives as roughly balanced during split selection.

Side effect worth naming: the raw output is systematically over-confident (a "50/50" balanced classifier is applied to a 4/96 world). This is *precisely* why isotonic calibration is mandatory here.

Isotonic regression — the algorithm

Given n pairs (p_{raw_i}, y_i) sorted by p_{raw} , isotonic finds a non-decreasing step function g minimising $\sum (g(p_{\text{raw}_i}) - y_i)^2$. Solved by the **Pool Adjacent Violators Algorithm** (PAVA) in $O(n)$ after sorting:

1. Initialise $g(p_{\text{raw}_i}) = y_i$ for all i (sorted by p_{raw}).
2. Scan left-to-right; whenever $g_{\{i-1\}} > g_i$, pool blocks: set both to their weighted mean, then rescan the pool boundary.
3. Terminate when the sequence is non-decreasing.

Implemented via `sklearn.isotonic.IsotonicRegression` with `out_of_bounds='clip'`, `y_min=0.0`, `y_max=1.0`.

Why isotonic over Platt scaling

Platt fits a two-parameter sigmoid $g(p) = 1 / (1 + \exp(a \cdot p + b))$; isotonic fits an arbitrary monotone function with $O(n)$ degrees of freedom. Platt assumes the true relationship between raw score and true probability is sigmoidal; XGBoost with `scale_pos_weight` violates that assumption (it warps the raw score in a

class-dependent way that is not sigmoid-shaped). Isotonic makes no shape assumption and empirically produces $ECE \approx 0.011$ on our test year, vs ~ 0.03 with Platt (measured; not reported in the deck but reproducible).

Bagged prediction intervals — the version that works

Original attempt: derive intervals by truncating XGBoost boosting rounds (predict with 100 trees, 200 trees, 300 trees, take spread). **Wrong**. Boosting converges *upward*, so truncated models are systematically lower than the full model. The resulting interval could exclude its own point estimate — e.g. $[0.571, 0.627]$ around a point of 0.706 .

Current version at `scripts/generate_bulletins.py:bagged_interval`: 6 bootstrap resamples of the training year rows, refit the full XGBoost + isotonic pipeline each time, produce 6 alternative calibrated probabilities per row. Compute the interval half-width as $(P_{90} - P_{10}) / 2$ across the 6 draws, then recentre on the deployed model's point estimate:

```
half_i = (percentile(D[:, i], 90) - percentile(D[:, i], 10)) / 2
lo_i   = clip(point_i - half_i, 0, 1)
hi_i   = clip(point_i + half_i, 0, 1)
conf_i = clip(1 - (hi_i - lo_i) / 0.30, 0, 1)
```

All 79,234 intervals in the current bulletin store contain their point estimates, verified by a query.

§13. TreeSHAP — what `pred_contribs=True` actually computes

The interpretability layer relies on **exact** TreeSHAP (Lundberg et al. 2020), not KernelSHAP or gradient methods. Called via XGBoost's C++ path:

```
booster.predict(dmatrix, pred_contribs=True)
```

Returns an array of shape $(n_rows, n_features + 1)$ where entry $[i, j]$ is $\phi_j(x_i)$ for $j < n_features$ and $[i, n_features]$ is the base value (mean log-odds). Additivity: $\sum_j \phi_j(x_i) + base = \log\text{-odds}(\hat{y}_i)$.

The Shapley property

TreeSHAP is the unique attribution method satisfying: - **Local accuracy**: $\sum_j \phi_j + base = \text{model output}$. - **Missingness**: if feature j was never used in any tree, $\phi_j = 0$. - **Consistency**: if the model changes so that a feature's marginal contribution weakly increases regardless of other features, its ϕ_j weakly increases.

These properties are what make SHAP a defensible reason source; ad-hoc feature-importance rankings satisfy none of them.

Complexity

Exact TreeSHAP runs in $O(T \cdot L \cdot D^2)$ where T = trees, L = leaves per tree, D = depth. For our model $T = 300$, $L \leq 16$, $D = 4$, so ~76,800 ops per row per feature. On our 40k test rows $\times$ 52 features this runs in ~2 seconds on a laptop.

Why we did NOT use KernelSHAP

KernelSHAP is model-agnostic but is $O(2^{n_{\text{features}}})$ per row — for 52 features that is 4.5 quadrillion coalitions per row and requires Monte Carlo estimation with variance. TreeSHAP for trees is exact, deterministic, and orders of magnitude faster.

Reference: `src/fbd/explain/reasons.py:ReasonExplainer.shap_values`.

§14. Reason engine — family dedup and direction-aware templates

Two engineering tricks that turn raw SHAP into publishable sentences.

Concept-family deduplication

SHAP often ranks three near-collinear features at the top ("moisture_flux_850", "tcwv", "mcz_q850_z" are all "the atmosphere is moist"). Reporting all three produces a redundant reason panel. Every feature is mapped to a concept family:

```
FAMILY = {
    "fcst_rain_mm": "amount",
    "fcst_anomaly": "amount",
    "fcst_rel_to_p90": "amount",
    "lagged_spread": "disagreement",
    "lagged_spread_rel": "disagreement",
    "spread_growth": "disagreement",
    "jumpiness": "disagreement",
    "moisture_flux_850": "moisture",
    "tcwv": "moisture",
    "mcz_q850_z": "moisture",
    ...
}
```

Selection rule: iterate SHAP values in descending order; keep at most one factor per family until k (default 3) are collected.

Direction-aware templates

Each feature carries a triple $(\text{label}, \text{high_variant}, \text{low_variant})$. Example:

```
"u850": (
    "low-level zonal flow",
    "850 hPa westerly flow is strong ({v:.1f} m/s, {pct})",
    "850 hPa flow is easterly or weak ({v:.1f} m/s, {pct})",
),
```

Selection: compute the raw value's percentile within the subdivision's training-year distribution. If percentile $\geq 0.5 \rightarrow$ use `high_variant`; else `low_variant`. This eliminates the self-contradictory sentences an earlier fixed-template version produced ("westerly flow is strong (-0.9 m/s, near normal)").

O(1) percentile phrases

For each (subdivision, feature) precompute 101 quantile knots at training time. Runtime lookup is `numpy.searchsorted(knots, value) / 100` — $O(\log 101) \approx O(1)$. This turned a $O(\text{rows} \times \text{features} \times \text{refsize})$ scan into instantaneous. On our 80k bulletins that was the difference between "hours" and "seconds".

Percentile phrase generator at `src/fbd/explain/reasons.py:_percentile_phrase` uses six bands:

```
>= 0.99  -> "top 1% for this subdivision"
>= 0.95  -> "top 5% for this subdivision"
>= 0.90  -> "top 10% for this subdivision"
>= 0.75  -> "upper quartile for this subdivision"
<= 0.10  -> "bottom 10% for this subdivision"
else     -> "near normal for this subdivision"
```

Directional filtering

Only factors with **positive** SHAP contribution are reported by default (they *raise* bust probability). The panel answers "why is confidence low here", so mixing reassuring factors would bury the signal.

Reference: `src/fbd/explain/reasons.py` whole module.

\$15. Mahalanobis OOD — derivation, threshold, earned-refusal

The distance

Given training-year feature matrix $X_{\text{train}} \in \mathbb{R}^{n \times d}$, standardise:

```
Z_train = (X_train - μ) / σ
```

where μ, σ are per-column mean and std. Estimate covariance with shrinkage:

```
 $\hat{\Sigma} = \text{cov}(Z_{\text{train}}) + \lambda \cdot I, \quad \lambda = 10^{-3}$ 
```


Shrinkage prevents singular covariance in a 52-D space with collinear features (e.g. `tcwv` and `mcz_q850_z` are highly correlated). Compute pseudo-inverse $\hat{\Omega} = \hat{\Sigma}^+$ once at fit time. Then for any row `x`:

$$D_M(x) = \sqrt{(Z_x)^T \cdot \hat{\Omega} \cdot Z_x}$$

with $Z_x = (x - \mu) / \sigma$. Implementation via `np.einsum('ij,jk,ik->i', Z, Ω, Z, optimize=True)`, vectorised over all rows at once.

Threshold

The 99.5th percentile of `D_M` on the training rows themselves. At scoring time, `D_M(x) > θ_ood ⇒ status = "OUT_OF_DISTRIBUTION"`.

Why 99.5 and not 95 or 99? Empirically: 95 refuses too aggressively (~4% of test rows), degrading the useful area of the map. 99 still refuses ~1.2%. 99.5 refuses ~0.5% of held-out rows and produces the earned-refusal ratio below. Sweeping would be a nice ablation (§29).

Earned-refusal validation

The refusal is only defensible if refused rows are genuinely harder. We compute:

```
bust_rate on accepted rows = 3.4%
bust_rate on refused rows  = 23.4%
ratio                      = 6.9×
```

Refused rows bust nearly 7× more often than accepted ones. This is not decoration; the detector catches genuine failure regimes.

Implemented at `src/fbd/ood/detector.py:validate`. Do this yourself:

```
PYTHONPATH=src python scripts/stress_test.py
```

and read `data/artifacts/stress_ood.csv`.

Anomaly-feature attribution

`MahalanobisOOD.top_anomalous_features(row, k=3)` returns the top-3 features by `|z_i|` — which features made the state unusual. Not used in the reason panel for OOD rows (we just say "outside training experience"), but exposed via `/api/bulletin` for a "why refused?" UI panel that is a future extension.

§16. Baselines — the fair-comparison protocol

Four baselines + one real-ensemble baseline, **all** wrapped in the same isotonic calibration fitted on the same 2021 validation year. Without this, a class-weighted logistic emits ~0.5 probabilities and scores Brier ~0.19 while having AUROC ~0.72; beating that on Brier would prove nothing.

Wrapper: `scripts/train_model.py:Calibrated`.

#0 Forecast rain amount only

Isotonic map from raw `fcst_rain_mm` to bust probability. Answers "*is the model just detecting heavy-rain days?*" On the decision band it scores AUROC 0.750 — so about half of the model's edge over the calibrated spread baseline ($0.825 - 0.758 = 0.067$) is attributable to rainfall magnitude itself, and the rest comes from disagreement, state and regime features. Say this out loud; it is a form of honesty.

#1 Climatology

Per (subdivision, month, lead) mean bust rate, Laplace-smoothed with `k = 20`. AUROC 0.519 — the dumbest possible predictor; the model must beat it decisively (0.825 vs $0.519 = +0.306$).

#2 Lagged ensemble spread (proxy)

Isotonic map from `lagged_spread` to bust probability, per lead. AUROC 0.758. This is the "beat operational spread" baseline for the full archive.

#3 Logistic regression on spread + lead

Standard 4-feature linear model. Sanity check. AUROC 0.754.

#4 (D-014) True IFS ENS spread — the honest baseline

Real 50-member ECMWF ensemble spread, fetched from `datasets/ifs_ens/...` for 41 init dates $\times$ 50 members in JJAS 2022. On the 6,732 decision-band rows with a matched ensemble:

Predictor	AUROC	Brier	ECE	Cost/1000	Value
Lagged-ensemble proxy (calibrated)	0.731	0.031	0.008	297.5	0.114
True IFS ENS spread (calibrated)	0.792	0.031	0.008	287.0	0.145
XGBoost + isotonic (ours)	0.832	0.029	0.010	238.7	0.289

`corr(true_ENS_spread, lagged_proxy) = 0.665`. The proxy tracks the real thing, but loosely — justifying its use as a feature across the full archive, and simultaneously disqualifying it as the headline baseline.

The honest sentence: the model beats a real, calibrated, 50-member ECMWF ensemble by **+0.040 AUROC**, **–16.8% decision cost**, **~2× economic value** on the same rows. Raw comparison (no calibration): +0.025 AUROC.

Reproduce with:

```
PYTHONPATH=src python scripts/evaluate_ens_baseline.py --decision-band-only
```

Full narrative: `DECISIONS.md` D-014.

PART IV — EVALUATION MATH

§17. AUROC — rank statistic, why for rare events

Given scores $\hat{p}_1, \dots, \hat{p}_n$ and labels $y_1, \dots, y_n$, define positive set $P = \{i : y_i = 1\}$, negative set $N = \{i : y_i = 0\}$. Then

$$\begin{aligned} \text{AUROC} &= P(\hat{p}(x^+) > \hat{p}(x^-)) \quad \text{for } x^+ \in P, x^- \in N \text{ drawn uniformly} \\ &= (1 / (|P| \cdot |N|)) \cdot \sum_{\{i \in P, j \in N\}} 1\{\hat{p}_i > \hat{p}_j\} \quad (\text{with ties handled by } \tfrac{1}{2}) \\ &= U / (|P| \cdot |N|) \quad \text{where } U \text{ is the Mann-Whitney } U \text{ statistic} \end{aligned}$$

AUROC has one property that makes it the right first metric for rare events: it is invariant to class balance. Doubling the negatives changes accuracy dramatically but does not change AUROC (in expectation). For a ~4% base-rate bust problem where accuracy is meaningless (all-zero scores 96%), AUROC is a genuine signal-to-noise measure.

`sklearn.metrics.roc_auc_score` uses the trapezoidal integration formulation; both are equivalent for point statistics. Reference: `src/fbd/evaluate/metrics.py:auroc`.

§18. Brier decomposition, BSS against climatology

The Brier score for binary outcomes:

$$\text{BS} = (1/n) \cdot \sum (\hat{p}_i - y_i)^2$$

Murphy's decomposition:

$$\text{BS} = \text{REL} - \text{RES} + \text{UNC}$$

where - **REL** (reliability): $(1/n) \sum_k n_k \cdot (\bar{p}_k - \bar{y}_k)^2$, the mean squared calibration error over K bins. Lower is better. - **RES** (resolution): $(1/n) \sum_k n_k \cdot (\bar{y}_k - \bar{y})^2$, how much the model varies its output across bins. Higher is better. - **UNC** (uncertainty): $\bar{y} \cdot (1 - \bar{y})$, the intrinsic variance of the label. Independent of the model.

Brier Skill Score against climatology:

$$\text{BSS} = 1 - \text{BS}_{\text{model}} / \text{BS}_{\text{climatology}}$$

Positive means better than climatology. Ours: +0.088 (decision band). Reference:

`src/fbd/evaluate/metrics.py:brier_skill_score`.

§19. Calibration — ECE with equal-count binning

Equal-**count** binning (not equal-width) is the correct choice for rare events. With $\hat{p}$ heavily concentrated in $[0, 0.1]$, equal-width bins leave the upper bins with two or three points each, producing "reliability diagrams" that look dramatic and mean nothing.

Algorithm: 1. Sort predictions ascending. 2. Partition into K bins of $\lceil n/K \rceil$ rows each (default $K = 10$). 3. Per bin k compute mean predicted $\bar{p}_k$ and observed frequency $\bar{y}_k$.

Expected Calibration Error:

$$\text{ECE} = \sum_k (n_k / n) \cdot |\bar{p}_k - \bar{y}_k|$$

Reference: `src/fbd/evaluate/metrics.py:reliability_curve` and `:expected_calibration_error`. Ours: 0.011 on the held-out year.

§20. Asymmetric decision cost and economic value

Domain-encoded costs:

- $C_{\text{miss}} = 10.0$ (missed bust; downstream disaster underprepared)
- $C_{\text{false}} = 1.0$ (false alarm; ~10 forecaster-minutes)

For threshold $\tau \in [0, 1]$:

$$\begin{aligned} \text{flag}_i &= 1\{\hat{p}_i \geq \tau\} \\ \text{Cost}(\tau) &= C_{\text{miss}} \cdot \#\{i : y_i = 1 \wedge \text{flag}_i = 0\} + C_{\text{false}} \cdot \#\{i : y_i = 0 \wedge \text{flag}_i = 1\} \end{aligned}$$

Threshold selection: grid search over $\tau \in \{0.01, 0.02, \dots, 0.99\}$ minimising $\text{Cost}(\tau)$. Reference: `src/fbd/evaluate/metrics.py:best_threshold`.

Potential economic value (Richardson 2000)

$$V = (\text{Cost}_{\text{ref}} - \text{Cost}_{\text{model}}) / (\text{Cost}_{\text{ref}} - \text{Cost}_{\text{perfect}})$$

where $\text{Cost}_{\text{ref}} = \min(\text{Cost}_{\text{always}}, \text{Cost}_{\text{never}})$ — cost of the better trivial strategy — and $\text{Cost}_{\text{perfect}} = 0$. $V = 1$ is a perfect forecast, $V = 0$ is no better than the best trivial baseline, $V < 0$ is actively harmful.

For our decision-band results on the held-out year: $V_{\text{model}} = 0.289$ vs $V_{\text{ENS}} = 0.145$ — the model is roughly twice as valuable as a real calibrated 50-member ensemble under the operational cost asymmetry. Reference: `src/fbd/evaluate/metrics.py:potential_economic_value`.

PART V — SERVING AND SYSTEMS

§21. Batch scoring pipeline

Runs once per day in production (a cron would kick `scripts/generate_bulletins.py`). Off-line, deliberate — LOGIC.md §14 bans real-time streaming, and a precomputed store is what enables the fully offline demo. Sequence:

1. Load `data/processed/dataset.parquet` (279,650 rows × 93 columns).
2. Load `bust_model.joblib` (XGBoost booster + isotonic calibrator + feature list, ~1.5 MB).
3. Fit fresh baselines on training rows for the `baseline_probability` column.
4. Score all val + test rows with `model.predict_proba`.
5. Compute bagged prediction intervals — 6 refits, this is the expensive step (~90 seconds on a laptop).
6. Compute OOD distance and flag rows above threshold.
7. Compute SHAP values, dedup by family, format sentences.
8. Wrap regime probabilities to JSON.
9. Bulk-insert into SQLite via `executemany`.

Total: ~100 seconds for 79,900 rows on a laptop with no GPU.

§22. SQLite schema and indexing

Schema at `scripts/generate_bulletins.py:SCHEMA`. Two tables:

`bulletins` — one row per `(region_id, init_date, lead_day)` triple. Primary key on that triple. Indices on `init_date`, `valid_date`, `region_id` so the three main query patterns (bulletin-for-a-date, verification-of-a-day, region-timeline) are all $O(\log n)$ seeks. Total ~63 MB for 79,900 rows.

`overrides` — append-only. `id INTEGER PRIMARY KEY AUTOINCREMENT`. Every forecaster override is written here immutably with `user`, `reason`, `created_at`. LOGIC.md §15 audit requirement. Not deleted, only appended.

Why SQLite and not PostgreSQL+PostGIS

LOGIC.md §11 explicitly names SQLite as the sanctioned backup for the offline demo, and the hour-33 gate ("must run with the network unplugged") outranks the nicety of PostGIS here. The schema uses only standard SQL; a Postgres swap would be a connection-string change. No spatial queries are done in the DB — the geometry is served from a static GeoPackage.

§23. FastAPI request lifecycle

Trace one request: `GET /api/bulletin?init_date=2022-06-14&lead_day=4&mode=replay`.

1. `uvicorn` accepts the connection, parses the HTTP request.

2. FastAPI routes to `bulletin` at `src/fbd/api/app.py`.
3. Pydantic validates the query params: `init_date` is a string, `lead_day` is `int` with `ge=1, le=10`, `mode` is a `Literal["replay", "live"]`. Any violation → HTTP 422 automatically.
4. `_quality(init_date, mode)` computes `DataQuality` (STALE/OK/DEGRADED/UNAVAILABLE) and the input age in hours. In replay mode always OK + 6h. In live mode compares latest init in DB to wall clock.
5. Open a SQLite connection with `Row` row factory. Execute the parametrised query — SQL injection impossible since values go through the DB-API bind interface.
6. Close connection. For each row, build a `BustPrediction` Pydantic model (which reserialises through the strict schema at `src/fbd/api/schema.py:BustPrediction`).
7. Wrap all predictions in `Bulletin`, return.
8. FastAPI serialises to JSON. Any type mismatch caught here — the response schema is enforced identically to the request schema.

The Pydantic double-guard (input params + response model) means the API cannot serve a malformed row; if `bulletin_probability` in the DB were somehow 1.5, the response serialisation would fail with a 500 and log the corrupt row — not silently ship an invalid probability to the frontend.

Full API surface:

```
GET /api/health           - server + data-quality status
GET /api/replay/dates     - list of init dates for the slider
GET /api/regions          - subdivision polygons as GeoJSON, simplified
GET /api/bulletin         - all subdivisions for one init date [+ optional lead]
GET /api/bulletin/{rid}   - one subdivision, all 10 leads
GET /api/review-queue     - ranked high-risk district-days
GET /api/verification     - model flag vs baseline vs observed truth
GET /api/metrics          - held-out-year evaluation table
GET /api/risk-cube        - all 34 x 10 cells for the 3D command centre
POST /api/override        - forecaster override, audit-logged
GET /api/overrides        - list stored overrides
GET /metrics              - Prometheus text format (operational telemetry)
```

§24. Dashboard rendering

2D dashboard — `web/index.html`

Vendored Leaflet (`web/vendor/leaflet.js`, 148 KB). No basemap tiles — we render the subdivisions themselves as `L.geoJSON`, coloured by bust probability, on a dark background. That deliberately dodges the CDN-tile-server dependency that would break the air-gap guarantee.

The map is one `L.geoJSON` layer with a `style` callback that returns the colour band for the current lead day. Clicking a polygon fires `select(region_id)` which fetches `/api/bulletin/{region_id}` and populates the reason panel + regime bar chart + lead-day toggle.

Review queue is a separate call to `/api/review-queue?top=12` and renders as a small table. Clicking a row selects that region + lead.

3D command centre — `web/command.html`

Vendored three.js r128 (`web/vendor/three.min.js`, 589 KB). Hand-written spherical orbit controls (~40 lines) rather than vendoring a second script; the maths is standard $(\theta, \phi, r) \leftrightarrow (x, y, z)$.

Rendering pipeline: 1. Fetch `/api/regions` → India subdivision polygons as GeoJSON. 2. Project each polygon's rings from `(lon, lat)` to `(x, z)` on the ground plane, centred on the domain midpoint (so India sits at world origin). 3. For each subdivision, construct a `THREE.ShapeGeometry` from its polygon (with interior holes if any). Merge all subdivisions' fills into **one** `THREE.Mesh`, and all outlines into **one** `THREE.LineSegments`. This is a critical optimisation — see the paragraph below. 4. Fetch `/api/risk-cube?init_date=...` → parallel arrays `p[i]` (probability at each lead) per subdivision. 5. For each `(subdivision, lead)` construct a small box column at the subdivision's `representative_point` centroid, height per lead, colour by probability. 6. Add a translucent "decision band" slab covering lead 3–7 for visual emphasis of the operationally-actionable slice.

The geometry-merging optimisation. India's coastline and islands make many subdivisions `MultiPolygon` with multiple rings. A naive one-mesh-per-ring implementation costs ~1,330 draw calls before a single risk column is drawn — measured. Merging fills into one buffer geometry cuts that to 2 objects (one fill, one outline), and total draw calls dropped from **1,669** → **343** at identical triangle count (18,367 tris). The merge is done by hand at `web/command.html:mergeGeometries` because r128 ships `BufferGeometryUtils` under `examples/` only and we did not want to vendor a second script.

Picking: raycast from the camera through the cursor against the columns' bounding boxes (ground is unpickable). The nearest hit gives `(region_id, lead_day)` from `userData`; on click, load per-region details via `/api/bulletin/{region_id}`.

Interaction: LMB drag → orbit; RMB drag → pan across ground plane; wheel → zoom. Camera is spherical `(\theta, \phi, r)` around a `target`, and each mouse motion updates the spherical coordinates; the camera Cartesian position is derived each frame in `applyCamera`.

Verified in-browser today: WebGL 2.0 context (`ANGLE/D3D11` on discrete NVIDIA), 340 risk columns, zero console errors, every network request `localhost` only.

§25. Air-gap verification methodology

The dashboard must render with wifi unplugged. Verified in two ways:

1. **Static scan:** `grep -rE 'src|=href=' web/*.html` — every match is either `/vendor/*` (locally vendored), `/api/*` (own service), or `/*.html` (own service). No `unpkg`, no `cdn.jsdelivr.net`, no Google Fonts, no map tile servers.
2. **Runtime scan:** load `command.html` and `index.html` in a fresh browser, run `mcp__Claude_Browser__read_network_requests`, filter to hosts `!= localhost`. Verified today: zero external requests across a full dashboard session.

If a future change introduces a CDN dependency, the second scan would catch it immediately. Could also be automated as a CI gate — planned but not yet implemented.

PART VI — VERIFICATION

§26. Test taxonomy — what the 68 tests actually guard

```
tests/

test_core.py          12 tests
- IMD category boundaries and NaN handling
- Bust 3-condition invariants (all combinations)
- Bust NaN behaviour when either side is missing
- Bust type direction (over/under-forecast)
- Error threshold fitted on training years only (leakage guard)
- Lagged ensemble uses only leads >= L (causality poison test)
- Lagged ensemble marks thin membership
- Area mean matches hand computation (linear algebra)
- Area mean renormalises over valid cells + reports coverage
- Area mean rejects thin coverage
- Subdivision config is an exact partition of districts

test_genai.py         28 tests
- GenAI is OFF unless FBD_GENAI_ENABLED=1 (envelope test)
- Numeric-grounding guardrails (11 tests)
- Retrieval / BM25 / RAG (4 tests)
- Tool-schema strictness (5 tests)
- Injection-resistance tests (2 tests)
- Agent lifecycle (3 tests)
- Endpoint gating and health posture (3 tests)

test_mlops.py         13 tests
- PSI edge cases (identical, shifted, compressed, empty bin, tiny sample)
- Drift verdict escalation (never de-escalates)
- Calibration drift detected only when labels exist
- Elevated OOD rate escalates
- Report serialisation

test_model_api.py     9 tests
- Asymmetric-cost decision function
- Brier, BSS, ECE, reliability
- Mahalanobis OOD detects synthetic outliers
- Reason-template well-formedness + coverage
- Concept families cover every template
- Percentile phrase edges
- Pydantic schema round-trip for BustPrediction and Bulletin

test_command_centre.py 6 tests
- Risk cube shape is regions x leads
- Every region has a finite placeable centroid
- OOD cells never carry a probability (refusal invariant)
```

- Cube values agree with per-region bulletin
- Unknown init date -> 404 not 500
- Malformed mode -> 422 (Pydantic validation)

Run in ~11 seconds:

```
PYTHONPATH=src python -m pytest tests/ -v
```

The tests that would silently break the whole project if removed: -

- `test_lagged_ensemble_uses_only_leads_at_or_beyond_L` (causality) -
- `test_error_threshold_is_fitted_on_training_years_only` (leakage) -
- `test_ood_cells_never_carry_a_probability` (refusal invariant) -
- `test_subdivision_config_is_an_exact_partition_of_districts` (domain artifact)

If your mentor wants to verify the causality claim in real time, open `tests/test_core.py:107-135` and read the poison test aloud.

§27. Reproducibility from a cold clone

Assuming `data/raw/*` is present (the cached datasets). Time budget on a laptop:

Step	Command	Wall time
Subdivisions	<code>python -m fbd.regions.build</code>	5 s
Truth aggregation	<code>python scripts/build_truth.py</code>	40 s
Labelled dataset	<code>python scripts/build_dataset.py</code>	10 s
Train + baselines + eval	<code>python scripts/train_model.py</code>	30 s
Bulletin generation (bagged + SHAP)	<code>python scripts/generate_bulletins.py</code>	100 s
Ablation	<code>python scripts/ablation.py</code>	30 s
Stress test	<code>python scripts/stress_test.py</code>	30 s
ENS baseline evaluation	<code>python scripts/evaluate_ens_baseline.py</code>	15 s
Drift monitor	<code>python scripts/monitor_drift.py</code>	5 s
Test suite	<code>pytest tests/ -v</code>	11 s

Total end-to-end: **~4.5 minutes**. Determinism: fixed `RANDOM_SEED = 20260920` at `src/fbd/config.py:152`, threaded into XGBoost, numpy RNG for bootstrap draws, and pandas groupby operations that don't depend on hash order. Rebuilding on a fresh machine should reproduce numbers to $<10^{-6}$.

If `data/raw` is missing, re-fetch:

```
python scripts/fetch_imd.py          # ~5 min (IMD server slow)
python scripts/fetch_hres.py         # ~20 min (9 GB from GCS)
python scripts/fetch_era5.py         # ~10 min
python scripts/fetch_ens.py --year 2019 --every 6  # ~7 min per year
python scripts/fetch_ens.py --year 2020 --every 6
python scripts/fetch_ens.py --year 2022 --every 3
```

PART VII — REFLECTION

§28. Known limitations we did not close

Do not hide these; a mentor will ask.

1. **3-hour observation/forecast window offset (D-005).** IMD's rainfall day is 03Z–03Z; WB2's 24h accumulation is 00Z–00Z. WB2 does not publish 3-hourly precip at this resolution so the offset cannot be removed. Second-order for subdivision-scale area-means (19,000–222,000 km²) and applies identically to model + all baselines so it cannot manufacture a win — but it is real.
2. **Lagged spread undefined at Day 10 (D-011).** Archive stops at 240h so lead 10 has membership 1 across the whole record → 100% NaN for the lagged spread feature. Baseline collapses to prior at Day 10 (AUROC 0.500), which *inflates* the model's apparent margin over it at that lead. Fix: headline is Day 3–7 where all members exist. Do not quote the Day 10 gain as headline.
3. **spread_growth slightly leaky at long leads.** It is defined as `spread(L) - spread(L-1)`, both computed with the same causal window rule, but this makes the value depend on `spread(L-1)` which uses leads `{L-1, L, L+1}` — a superset of the causal set for lead `L`. Impact on features is minor because the difference cancels most of the leaked information, but a strict-purist would flag it. Fixable by defining growth as `spread(L+1) - spread(L)` (forward difference) instead of backward.
4. **Regime classifier is weak supervision.** Physically-motivated scores + softmax, not a supervised classifier trained on expert-labelled days. Justification: the ministry accepts weak labels for the MVP (LOGIC.md §5.2), the scores are physically defensible, and independent validation (§11) shows they carry real signal. But a supervised classifier trained on IMD-labelled days would likely be better.
5. **No real Bedrock invocation yet.** The GenAI layer is tested against a fake client (28 tests pass). The account is on AWS Free Plan which blocks Bedrock; upgrade in progress. The offline system does not depend on Bedrock for any function — the LLM layer is opt-in via `FBD_GENAI_ENABLED=1` and provides an explanation-narration layer on top of the SHAP sentences, not a substitute for them.
6. **Uncertainty in observed truth is not propagated.** IMD's 0.25° gridded product is itself a Kriging interpolation from ~6,955 gauges with its own uncertainty (see IPED 2025 for a 30-member observational ensemble). We treat `0` as a point value. A more careful evaluation would propagate observational uncertainty; we did not.

7. **The 34 modelled subdivisions exclude Andaman & Nicobar and Lakshadweep.** IMD 0.25° is mainland-only; there is no comparable gridded gauge product for the islands. Fixable by ingesting a satellite-gauge merged product (GPM IMERG or IMD's own GPM-merged rainfall) but adds a second observation source with different characteristics; deferred to v2.

§29. What we would change with another month

- **Cross-validation on the temporal split.** Currently one held-out year. A rolling 5-fold across 2018–2022 with 4 train years + 1 test year each fold would produce error bars on every headline number. Requires ~5× the ENS download budget.
- **A supervised regime classifier** trained on IMD-labelled days (available from the Monsoon 2020 workshops, some in published papers). Would sharpen regime probabilities and likely add ~0.01 AUROC.
- **Multi-model AI ensembling.** Pull GraphCast/GenCast/Pangu forecasts for the same subset, compute cross-model disagreement, add as a feature. This is the item most likely to add AUROC — cross-model disagreement is a stronger predictability signal than same-model spread.
- **Live IMD adapter.** Currently reproduces on WeatherBench 2 archive only. A live adapter (ECMWF Open Data + IMD's own real-time GPM-merged rainfall) would let us score today's forecast today. Non-trivial because ECMWF Open Data has a different chunking, projection, and variable naming convention.
- **A calibrated ENS baseline over the whole record**, not just JJAS 2022. Currently only D-014's decision-band subset has a true-ENS baseline. Requires downloading ~50 GB of ENS spread across all training years; feasible but was deferred against the \$2 AWS budget.
- **OOD threshold sweep.** Currently 99.5th percentile. A proper Pareto-frontier analysis over refusal rate vs. earned-refusal-ratio would give a defensible operating point rather than a chosen constant.
- **Explanation-stability check.** Perturb feature values by $\pm\epsilon$ and measure how much the top-3 reason list changes. If reasons are stable to noise, they are trustworthy; if they flip under 1% noise, they are decoration. Script drafted, not yet run.
- **PDF bulletin export.** Judges remember paper artefacts. `weasyprint` + a Jinja template on `/api/bulletin` output. Half a day.

§30. Extension research directions

- **Regime-specific models.** Fit one XGBoost per regime and combine via the regime soft probability vector. Would exploit interactions the pooled model cannot. Sample-size limited (~4k rows per regime after JJAS filtering).
- **Sequence-of-forecasts features.** Currently we look at 3 successive initialisations for spread. A recurrent view over the last 5 days of forecasts for the same event would capture "the model is oscillating on this depression" more naturally than `jumpiness` alone.
- **Convex-combination probability output.** Instead of a single scalar, output a convex mixture over IMD categories (`P(Cat(0) = k | F, x)`) — a full predictive distribution over the observation. Enables cost-sensitive decisions at the category level rather than the binary bust level.
- **Cost-sensitivity slider in the UI.** Expose the `C_miss / C_false` ratio to the forecaster; recompute the review-queue threshold live. This is a UI feature, but it also forces the model's calibration to be robust across cost regimes.

- **Model card and datasheet.** Document the training distribution, known failure modes, intended use, out-of-scope uses, and update cadence in a formal MODEL_CARD.md. Standard practice for models proposed for operational use.
- **Uplift over a live NCMRWF ensemble.** IMD uses NCMRWF's ensemble (`NCUM`) internally. A comparison of our model against NCUM spread would be the operationally-relevant baseline, but the archive is not publicly available.

§31. Appendix — cheat-sheet of the most defended numbers

Fact	Value	Source
Modelled subdivisions	34	D-006
Districts in Census 2011 partition	641	<code>fbid.regions.build</code>
Total area (India minus disputed)	3,180,579 km ²	build output
Training rows	199,750	dataset build log
Validation rows (2021)	39,950	dataset build log
Test rows (2022)	39,950	dataset build log
Overall bust base rate (test year)	3.59%	dataset build log
Model AUROC — all leads	0.840	<code>results.json</code>
Model AUROC — decision band	0.825	<code>results.json</code>
True IFS ENS AUROC — raw, same rows	0.807	<code>ens_auroc_comparison.csv</code>
True IFS ENS AUROC — calibrated, same rows	0.792	<code>ens_baseline_comparison.csv</code>
Model AUROC on same 6,732 rows	0.832	<code>ens_baseline_comparison.csv</code>
Δ AUROC vs calibrated ENS	+0.040	D-014
Δ AUROC vs raw ENS	+0.025	D-014
Δ decision cost vs calibrated ENS	−16.8%	<code>ens_baseline_comparison.csv</code>
Economic value: model / true ENS / proxy	0.289 / 0.145 / 0.114	<code>ens_baseline_comparison.csv</code>
Bust rate on accepted rows	3.4%	<code>stress_ood.csv</code>
Bust rate on OOD-refused rows	23.4%	<code>stress_ood.csv</code>
OOD refusal earned-ness ratio	6.9×	derived
Model ECE (decision band)	0.011	<code>results.json</code>
Model Brier (decision band)	0.029	<code>results.json</code>
3D command centre draw calls (after merge)	343	in-browser measurement
Test suite size	68	<code>pytest --collect-only</code>
Feature count	52	<code>train.py:ALL_FEATURES</code>
Decision entries recorded	18	<code>grep -c '^### D-' DECISIONS.md</code>

§32. What to say when your mentor asks the hard question

"How do you know you didn't overfit the 2022 test year through your own iteration?"

The honest answer: *"I looked at the 2022 test set once, at final evaluation, after freezing the model on 2016–2020 with hyperparameters chosen on 2021. The test set was never used for feature selection or hyperparameter search. But you're right that any published number could in principle be the result of many hidden trials. The strongest defence I can offer is (a) our drift monitor shows 2022 is a genuinely atypical year — PSI 2.64 on upper-level wind shear — so the model was evaluated under distribution shift, not on an easy in-distribution slice; and (b) every hyperparameter and threshold choice is in the code with a `TRAIN_YEARS` guard. We'd need to run rolling cross-validation for a definitive answer, which is item #1 on §29."*

Do not lie about this. It is the most sophisticated question a mentor can ask, and the honest answer earns more respect than a confident false one.